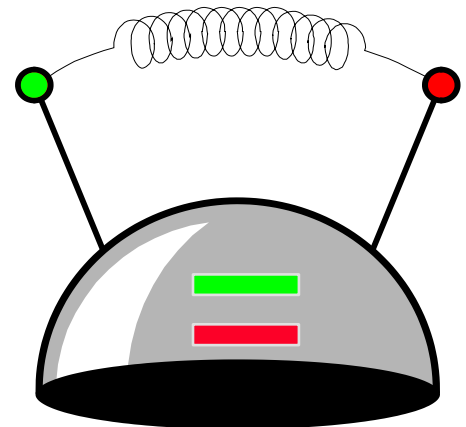


Objects and Classes

OO Programming Concepts

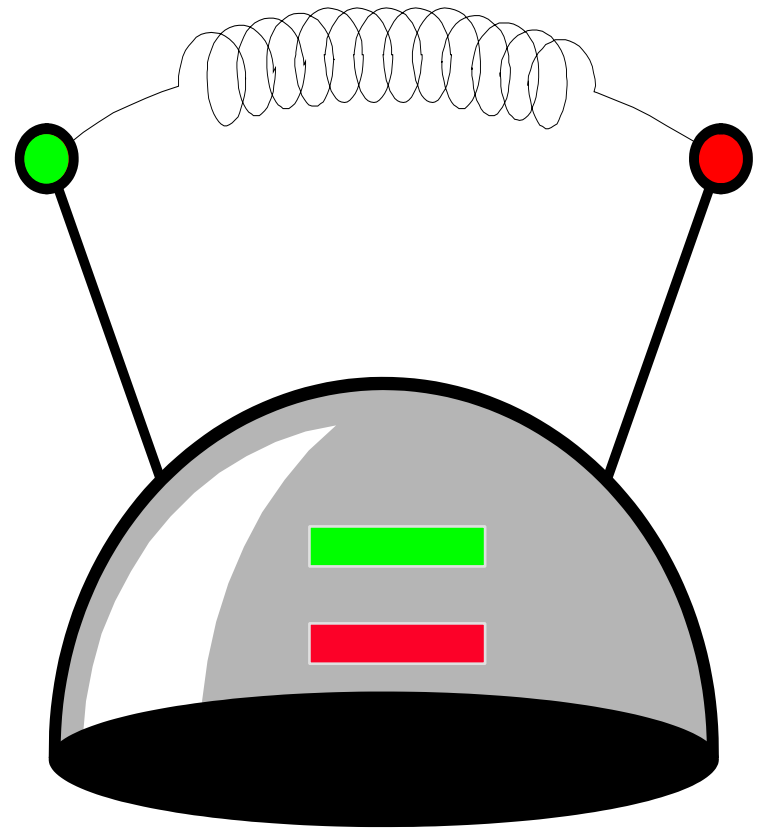
- ❑ OOP is an approach to programming which supports the **creation of new data types and operations to manipulate those types**.
- ❑ OOP involves programming using **objects**.

*Let's try to tell what is an object by **Thinking Cap** example*



Thinking Cap

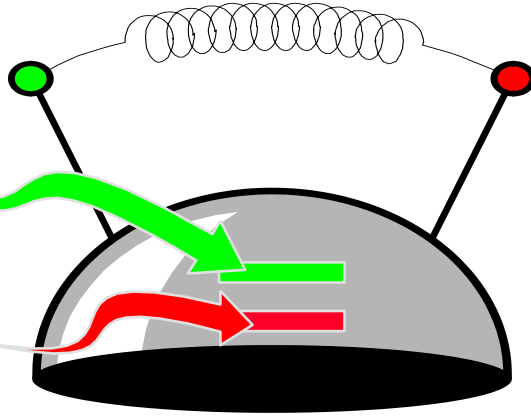
- ❑ In **Thinking Cap**, there are **two slots (green and red)**, with a sentence written on each.
- ❑ You may **push the green button** and the thinking cap will speak the sentence from the green slot's paper.
- ❑ And same for the **red button**.
- ❑ Assume we need two **Thinking Caps**, one for a student and the other for a football player



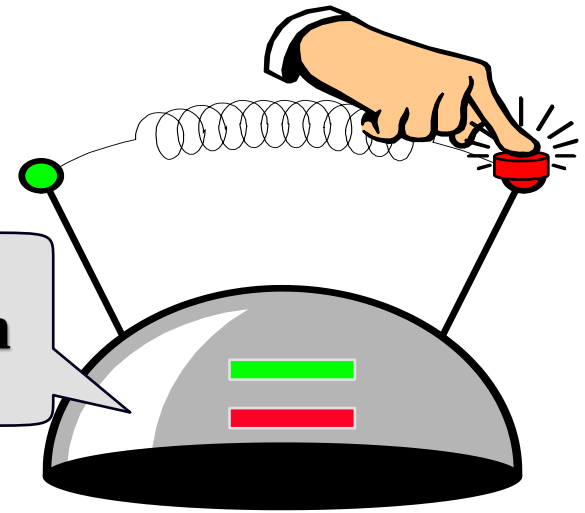
Student Thinking Cap

**Studying
means success**

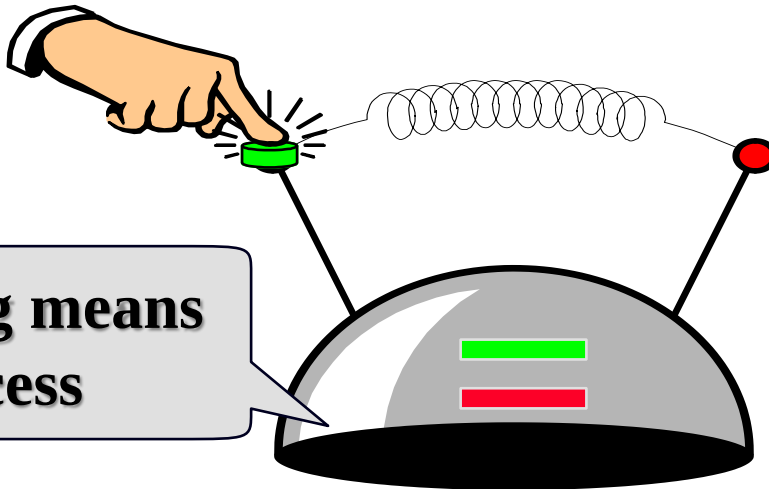
I have an Exam



I have an Exam



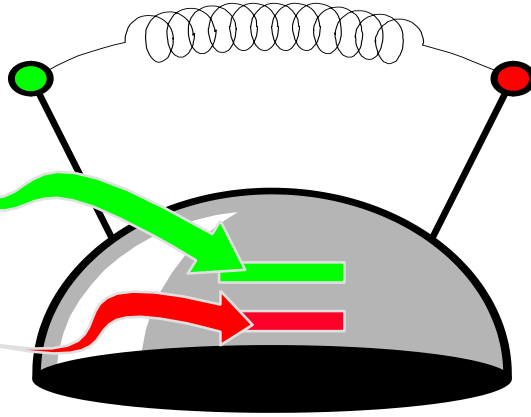
**Studying means
success**



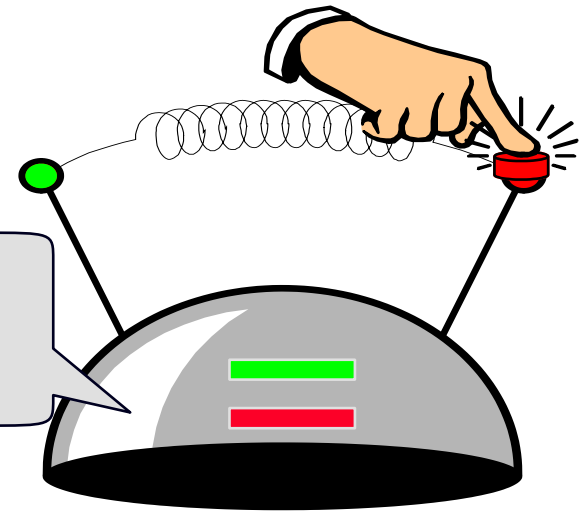
Player Thinking Cap

**Training means
win**

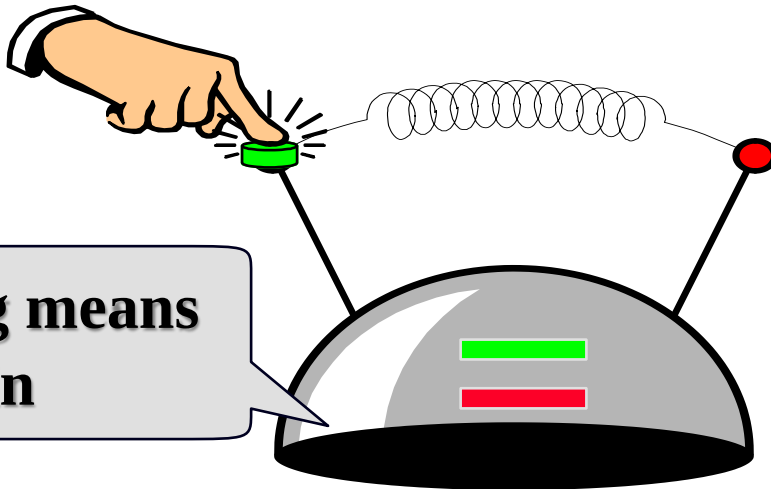
My feet hurt



My feet hurt

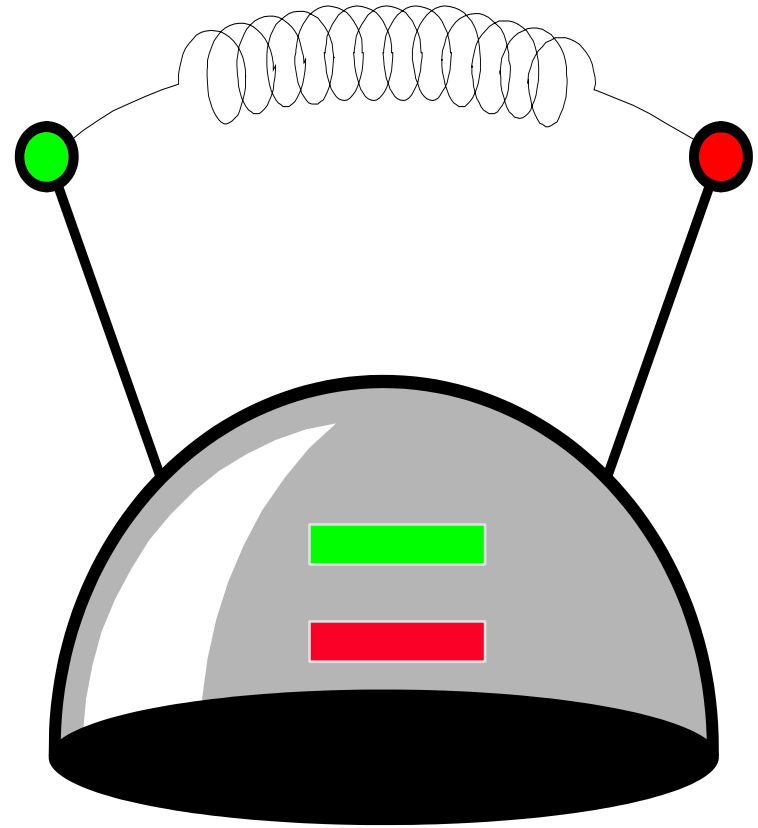


**Training means
win**

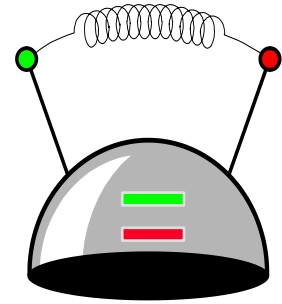


Thinking Cap

- We need first define what is the **Thinking Cap**.
- Then, we can use this definition to give **one** for a student and the another for a football player.



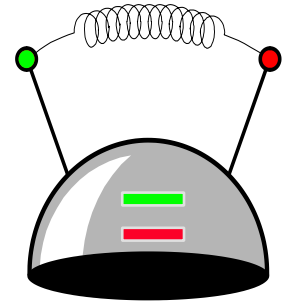
Thinking Cap



- We can describe the thinking cap within a class.

```
class Thinking_cap  
{  
  
    ...  
  
};
```

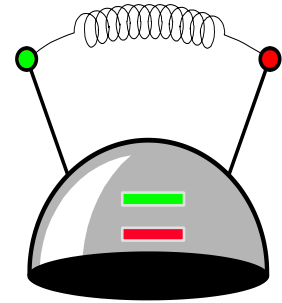
Thinking Cap



- The class will have two attributes called **green_string** and **red_string**. These attributes are strings which hold the information that is placed in the two slots.

```
class Thinking_cap
{
    ...
    String green_string;
    String red_string;
};
```

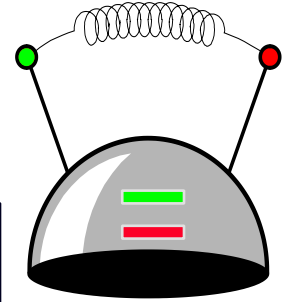

Thinking Cap



- Also, There are some operations which can be invoked on the Thinking Cap should be listed.

```
class Thinking_cap  
{  
    char green_string[50];  
    char red_string[50];  
  
    void fillSlots  
        (String greenMsg,  
         String redMsg);  
    void push_green( );  
    void push_red( );  
};
```

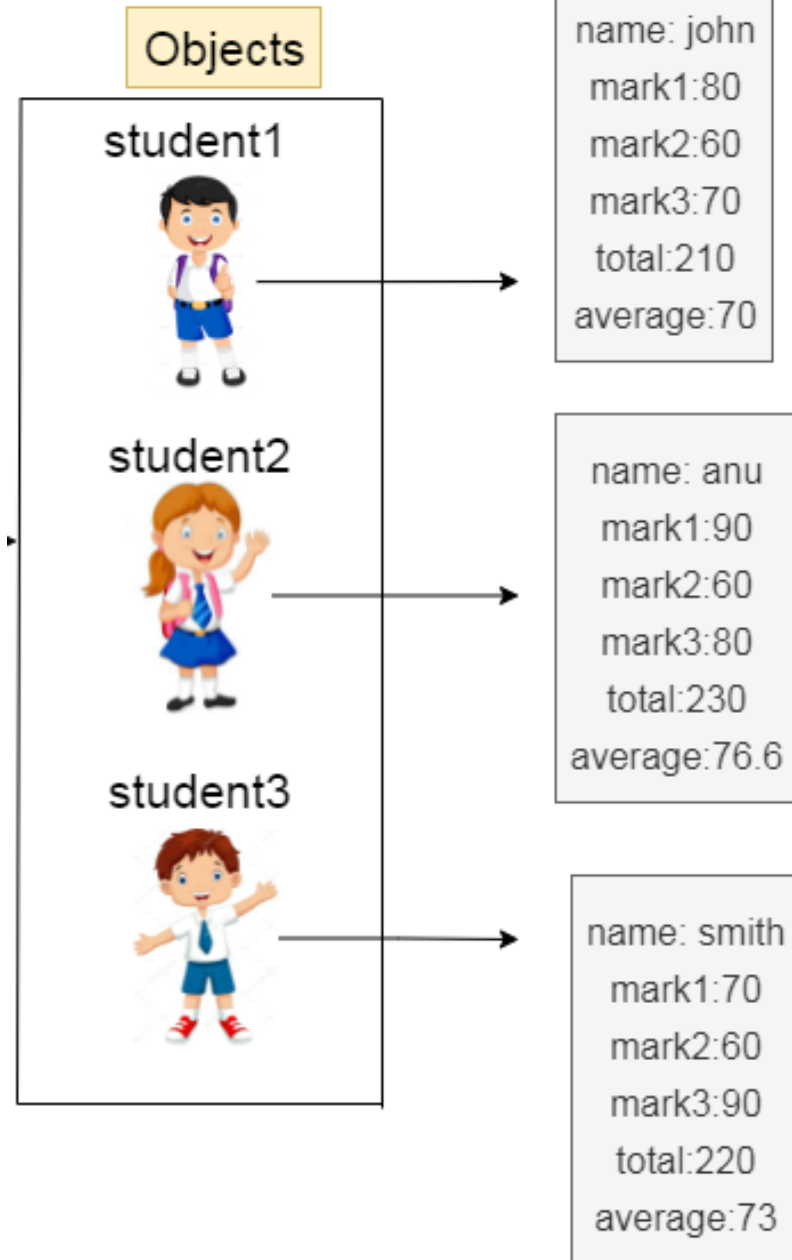
Using Thinking Cap



```
{  
  Thinking_cap studentCap;  
  Thinking_cap playerCap;  
  studentCap.fillSlots( "Hello", "Goodbye");  
  playerCap.fillSlots( "Haa", "Boo!");  
  student.push_green( );  
  student.push_red( );  
  ...  
}
```

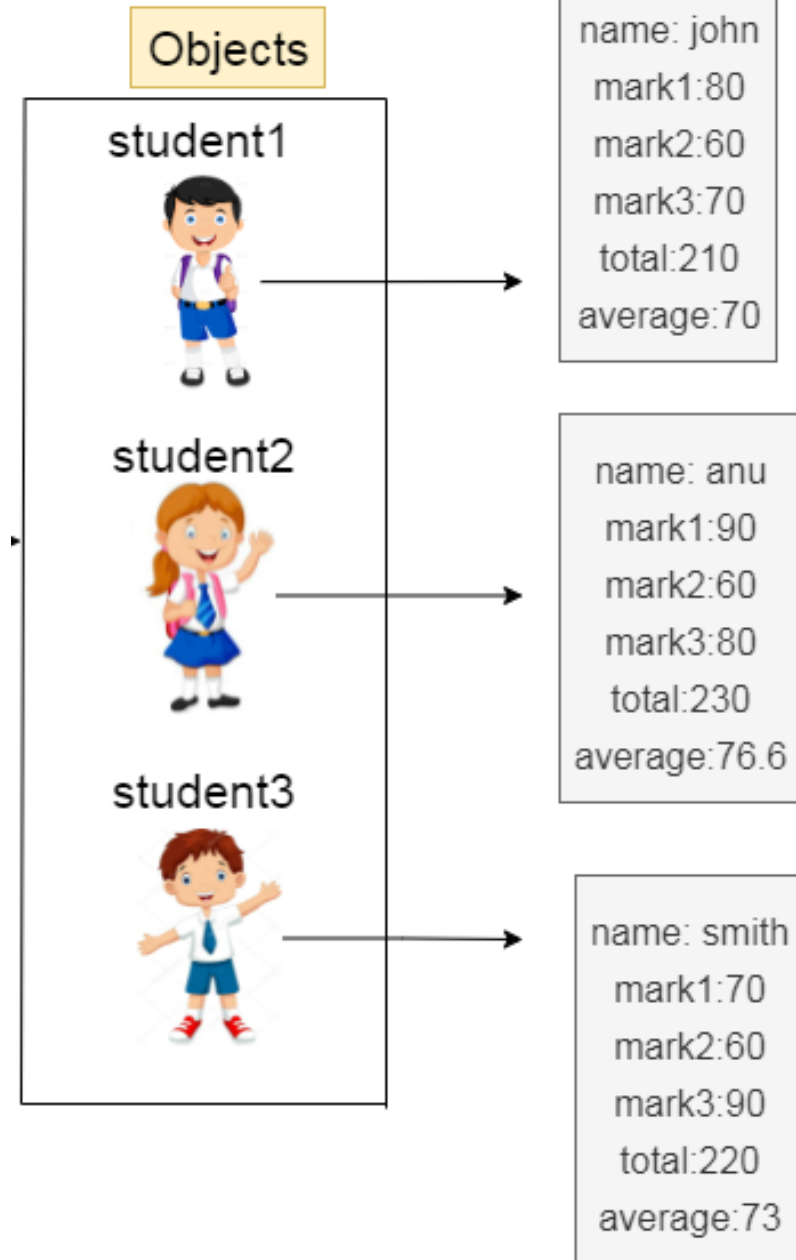
Objects

- An *object* represents an entity in the real world that can be distinctly identified.
- For example, a student, a desk, a circle, a button, and even a loan can all be viewed as objects.

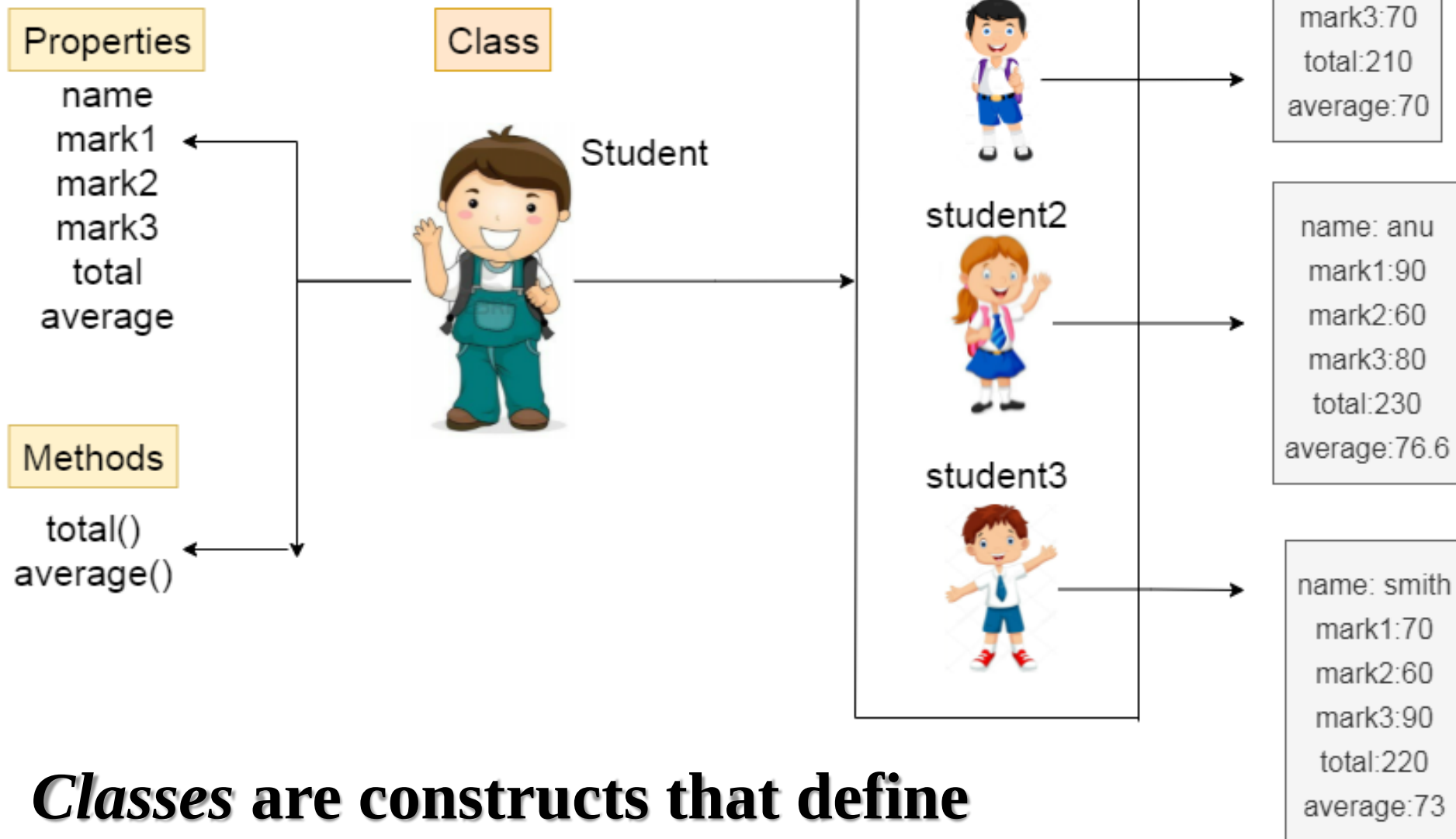


Objects

- An object has a unique identity, state, and behaviors.
- The *state* of an object consists of a set of *data fields* (also known as *properties*) with their current values.
- The *behaviors* describe what the object does or what is done on the object.



Classes



***Classes* are constructs that define objects of the same type.**

Classes

A Java class uses variables to define data fields and methods to define behaviors.

Additionally, a class provides a special type of methods, known as constructors, which are invoked to construct objects from the class.

Classes

```
class Circle {  
    /** The radius of this circle */  
    double radius = 1.0;  
  
    /** Construct a circle object */  
    Circle() {  
    }  
  
    /** Construct a circle object */  
    Circle(double newRadius) {  
        radius = newRadius;  
    }  
  
    /** Return the area of this circle */  
    double getArea() {  
        return radius * radius * 3.14159;  
    }  
}
```

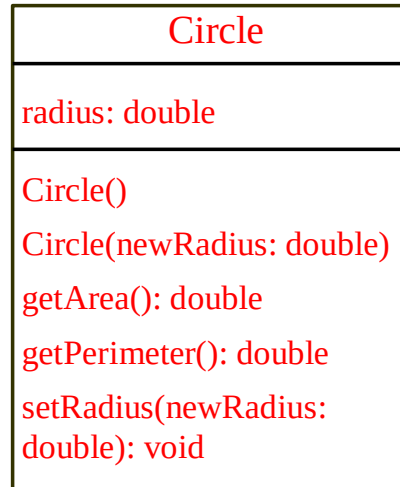
← Data field

← Constructors

← Method

UML Class Diagram

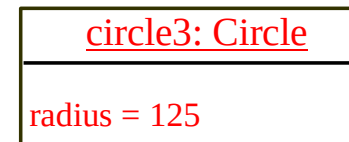
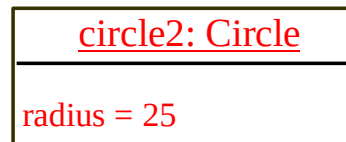
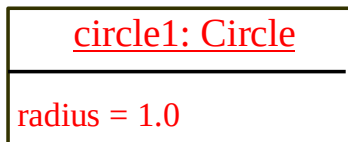
UML Class Diagram



← Class name

← Data fields

← Constructors and methods



← UML notation for objects

Example: Defining Classes and Creating Objects

TV	
channel: int volumeLevel: int on: boolean	
+TV() +turnOn(): void +turnOff(): void +setChannel(newChannel: int): void +setVolume(newVolumeLevel: int): void +channelUp(): void +channelDown(): void +volumeUp(): void +volumeDown(): void	

The + sign indicates
a public modifier. →

The current channel (1 to 120) of this TV.
The current volume level (1 to 7) of this TV.
Indicates whether this TV is on/off.

Constructs a default TV object.
Turns on this TV.
Turns off this TV.
Sets a new channel for this TV.
Sets a new volume level for this TV.
Increases the channel number by 1.
Decreases the channel number by 1.
Increases the volume level by 1.
Decreases the volume level by 1.

TV

TestTV

Example: Defining Classes and Creating Objects

Objective: Demonstrate creating objects, accessing data, and using methods.

TestSimpleCircle

Constructors

Constructors are a special kind of methods that are invoked to construct objects.

```
Circle() {  
}
```

```
Circle(double newRadius) {  
    radius = newRadius;  
}
```

Constructors, cont.

A constructor with no parameters is referred to as a *no-arg constructor*.

- Constructors must have the same name as the class itself.
- Constructors do not have a return type—not even void.
- Constructors are invoked using the new operator when an object is created. Constructors play the role of initializing objects.

Creating Objects Using Constructors

```
new ClassName() ;
```

Example:

```
new Circle() ;
```

```
new Circle(5.0) ;
```

Default Constructor

A class may be defined without constructors. In this case, a no-arg constructor with an empty body is implicitly defined in the class. This constructor, called *a default constructor*, is provided automatically *only if no constructors are explicitly defined in the class*.

Declaring Object Reference Variables

To reference an object, assign the object to a reference variable.

To declare a reference variable, use the syntax:

```
ClassName objectRefVar;
```

Example:

```
Circle myCircle;
```

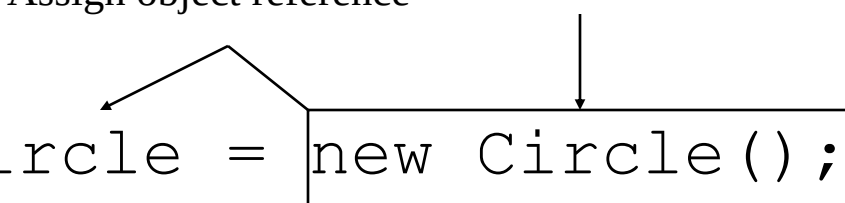
Declaring/Creating Objects in a Single Step

```
ClassName objectRefVar = new ClassName();
```

Example:

Assign object reference Create an object

```
Circle myCircle = new Circle();
```



Accessing Object's Members

- ❑ Referencing the object's data:

`objectRefVar.data`

e.g., `myCircle.radius`

- ❑ Invoking the object's method:

`objectRefVar.methodName (arguments)`

e.g., `myCircle.getArea()`

Trace Code

```
Circle myCircle = new Circle(5.0);
```

```
Circle yourCircle = new Circle();
```

```
yourCircle.radius = 100;
```

Declare myCircle

myCircle

no value

Trace Code, cont.

Circle myCircle = new Circle(5.0);

myCircle no value

Circle yourCircle = new Circle();

yourCircle.radius = 100;

<u>: Circle</u>
radius: 5.0

Create a circle

Trace Code, cont.

Circle myCircle = new Circle(5.0);

Circle yourCircle = new Circle();

yourCircle.radius = 100;

myCircle reference value

Assign object reference
to myCircle

: Circle

radius: 5.0

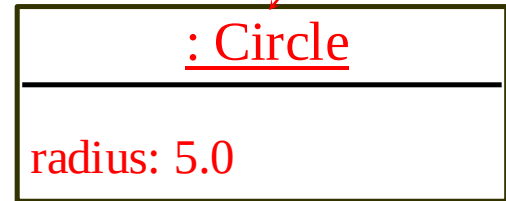
Trace Code, cont.

Circle myCircle = new Circle(5.0);

Circle yourCircle = new Circle();

yourCircle.radius = 100;

myCircle reference value



yourCircle no value

Declare yourCircle

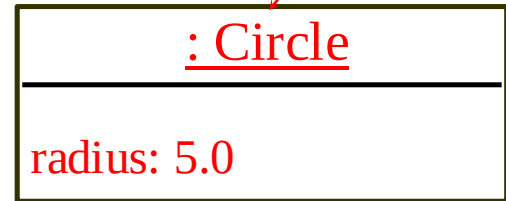
Trace Code, cont.

Circle myCircle = new Circle(5.0);

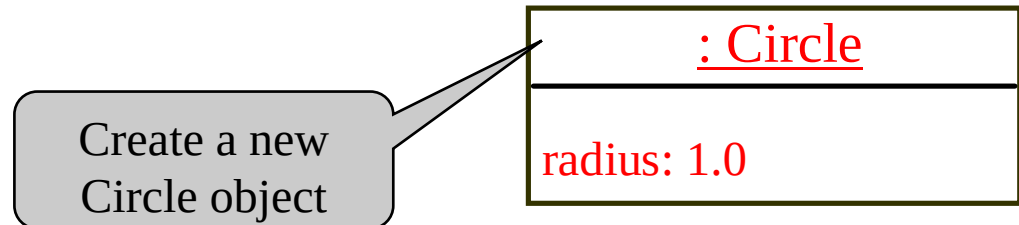
Circle yourCircle = new Circle();

yourCircle.radius = 100;

myCircle reference value



yourCircle no value



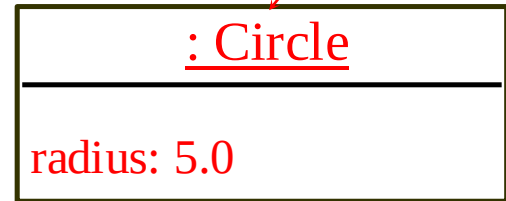
Trace Code, cont.

```
Circle myCircle = new Circle(5.0);
```

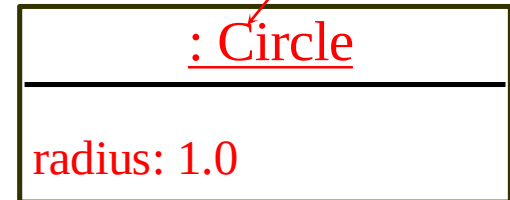
```
Circle yourCircle = new Circle();
```

```
yourCircle.radius = 100;
```

myCircle **reference value**



yourCircle **reference value**



Assign object reference
to yourCircle

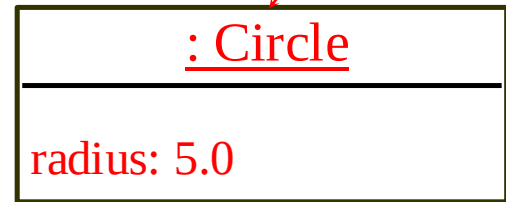
Trace Code, cont.

Circle myCircle = new Circle(5.0);

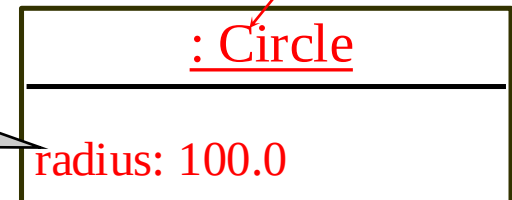
Circle yourCircle = new Circle();

yourCircle.radius = 100;

myCircle reference value



yourCircle reference value



Change radius in
yourCircle

Caution

Recall that you use

`Math.methodName(arguments)` (e.g., `Math.pow(3, 2.5)`)

to invoke a method in the `Math` class. Can you invoke `getArea()` using `SimpleCircle.getArea()`? The answer is no. All the methods used before this chapter are static methods, which are defined using the `static` keyword. However, `getArea()` is non-static. It must be invoked from an object using

`objectRefVar.methodName(arguments)` (e.g., `myCircle.getArea()`).

More explanations will be given in the section on “Static Variables, Constants, and Methods.”

Reference Data Fields

The data fields can be of reference types. For example, the following Student class contains a data field name of the String type.

```
public class Student {  
    String name; // name has default value null  
    int age; // age has default value 0  
    boolean isScienceMajor; // isScienceMajor has default value false  
    char gender; // c has default value '\u0000'  
}
```

The null Value

If a data field of a reference type does not reference any object, the data field holds a special literal value, null.

Default Value for a Data Field

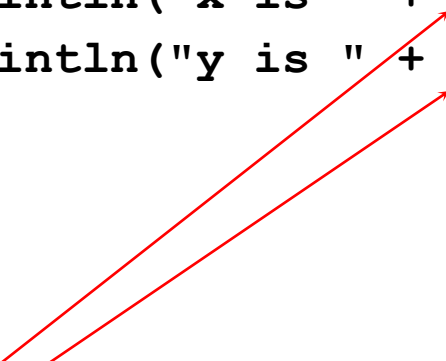
The default value of a data field is null for a reference type, 0 for a numeric type, false for a boolean type, and '\u0000' for a char type. However, Java assigns no default value to a local variable inside a method.

```
public class Test {  
    public static void main(String[] args) {  
        Student student = new Student();  
        System.out.println("name? " + student.name);  
        System.out.println("age? " + student.age);  
        System.out.println("isScienceMajor? " + student.isScienceMajor);  
        System.out.println("gender? " + student.gender);  
    }  
}
```

Example

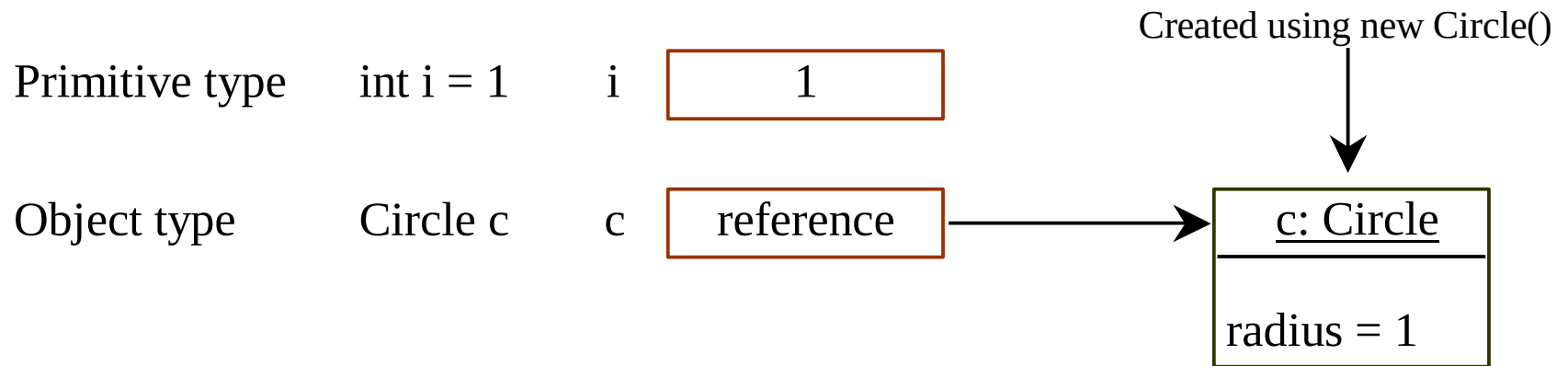
Java assigns no default value to a local variable inside a method.

```
public class Test {  
    public static void main(String[] args) {  
        int x; // x has no default value  
        String y; // y has no default value  
        System.out.println("x is " + x);  
        System.out.println("y is " + y);  
    }  
}
```



Compile error: variable not
initialized

Differences between Variables of Primitive Data Types and Object Types



Copying Variables of Primitive Data Types and Object Types

Primitive type assignment $i = j$

Before:

i

1

j

2

After:

i

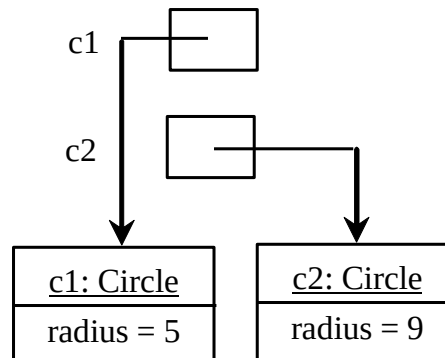
2

j

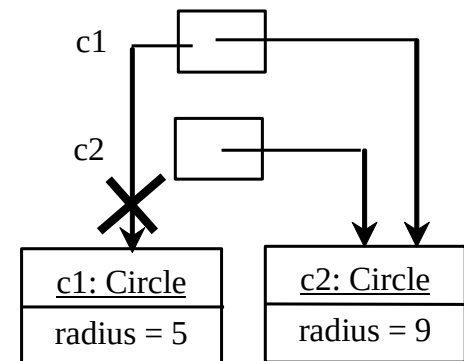
2

Object type assignment $c1 = c2$

Before:



After:



Garbage Collection

As shown in the previous figure, after the assignment statement `c1 = c2`, `c1` points to the same object referenced by `c2`. The object previously referenced by `c1` is no longer referenced. This object is known as garbage. Garbage is automatically collected by JVM.

Garbage Collection, cont

TIP: If you know that an object is no longer needed, you can explicitly assign null to a reference variable for the object. The JVM will automatically collect the space if the object is not referenced by any variable .

Visibility Modifiers and Accessor/Mutator Methods

By default, the class, variable, or method can be accessed by any class in the same package.

- ❑ `public`

The class, data, or method is visible to any class in any package.

- ❑ `private`

The data or methods can be accessed only by the declaring class.

The get and set methods are used to read and modify private properties.

```
package p1;

public class C1 {
    public int x;
    int y;
    private int z;

    public void m1() {
    }
    void m2() {
    }
    private void m3() {
    }
}
```

```
package p1;

public class C2 {
    void aMethod() {
        C1 o = new C1();
        can access o.x;
        can access o.y;
        cannot access o.z;

        can invoke o.m1();
        can invoke o.m2();
        cannot invoke o.m3();
    }
}
```

```
package p2;

public class C3 {
    void aMethod() {
        C1 o = new C1();
        can access o.x;
        cannot access o.y;
        cannot access o.z;

        can invoke o.m1();
        cannot invoke o.m2();
        cannot invoke o.m3();
    }
}
```

The private modifier restricts access to within a class, the default modifier restricts access to within a package, and the public modifier enables unrestricted access.

```
package p1;
```

```
class C1 {  
    ...  
}
```

```
package p1;
```

```
public class C2 {  
    can access C1  
}
```

```
package p2;
```

```
public class C3 {  
    cannot access C1;  
    can access C2;  
}
```

The default modifier on a class restricts access to within a package, and the public modifier enables unrestricted access.

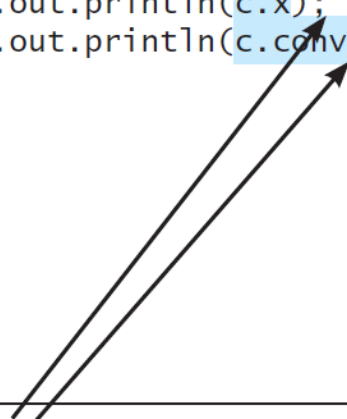
NOTE

An object cannot access its private members, as shown in (b). It is OK, however, if the object is declared in its own class, as shown in (a).

```
public class C {  
    private boolean x;  
  
    public static void main(String[] args) {  
        C c = new C();  
        System.out.println(c.x);  
        System.out.println(c.convert());  
    }  
  
    private int convert() {  
        return x ? 1 : -1;  
    }  
}
```

(a) This is okay because object `c` is used inside the class `C`.

```
public class Test {  
    public static void main(String[] args) {  
        C c = new C();  
        System.out.println(c.x);  
        System.out.println(c.convert());  
    }  
}
```



(b) This is wrong because `x` and `convert` are private in class `C`.

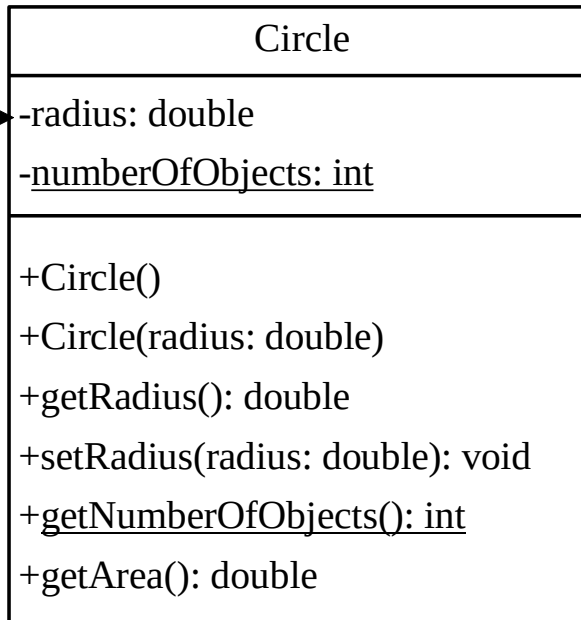
Why Data Fields Should Be private?

To protect data.

To make code easy to maintain.

Example of Data Field Encapsulation

The - sign indicates
private modifier



The radius of this circle (default: 1.0).

The number of circle objects created.

Constructs a default circle object.

Constructs a circle object with the specified radius.

Returns the radius of this circle.

Sets a new radius for this circle.

Returns the number of circle objects created.

Returns the area of this circle.

CircleWithPrivateDataFields

TestCircleWithPrivateDataFields

Scope of Variables

- ❑ The scope of instance and static variables is the entire class. They can be declared anywhere inside a class.
- ❑ The scope of a local variable starts from its declaration and continues to the end of the block that contains the variable. A local variable must be initialized explicitly before it can be used.

The this Keyword

- ❑ The this keyword is the name of a reference that refers to an object itself. One common use of the this keyword is reference a class's *hidden data fields*.
- ❑ Another common use of the this keyword to enable a constructor to invoke another constructor of the same class.

Reference the Hidden Data Fields

```
public class F {  
    private int i = 5;  
    private static double k = 0;  
  
    void setI(int i) {  
        this.i = i;  
    }  
  
    static void setK(double k) {  
        F.k = k;  
    }  
}
```

Suppose that f1 and f2 are two objects of F.
F f1 = new F(); F f2 = new F();

Invoking f1.setI(10) is to execute
 this.i = 10, where **this** refers f1

Invoking f2.setI(45) is to execute
 this.i = 45, where **this** refers f2

Calling Overloaded Constructor

```
public class Circle {  
    private double radius;
```

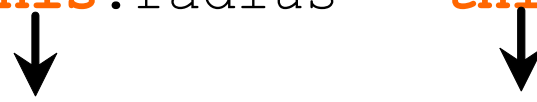
```
    public Circle(double radius) {  
        this.radius = radius;  
    }
```

 this must be explicitly used to reference the data field radius of the object being constructed

```
    public Circle() {  
        this(1.0);  
    }
```

 this is used to invoke another constructor

```
    public double getArea() {  
        return this.radius * this.radius * Math.PI;  
    }  
}
```

 Every instance variable belongs to an instance represented by this, which is normally omitted

The Date Class

Java provides a system-independent encapsulation of date and time in the java.util.Date class. You can use the Date class to create an instance for the current date and time and use its toString method to return the date and time as a string.

The + sign indicates
public modifier



java.util.Date	
+Date()	
+Date(elapseTime: long)	
+toString(): String	
+getTime(): long	
+setTime(elapseTime: long): void	

Constructs a Date object for the current time.

Constructs a Date object for a given time in milliseconds elapsed since January 1, 1970, GMT.

Returns a string representing the date and time.

Returns the number of milliseconds since January 1, 1970, GMT.

Sets a new elapse time in the object.

The Date Class Example

For example, the following code

```
java.util.Date date = new java.util.Date();  
System.out.println(date.toString());
```

displays a string like Sun Mar 09 13:50:19
EST 2003.

The Random Class

You have used Math.random() to obtain a random double value between 0.0 and 1.0 (excluding 1.0). A more useful random number generator is provided in the java.util.Random class.

java.util.Random	
+Random()	Constructs a Random object with the current time as its seed.
+Random(seed: long)	Constructs a Random object with a specified seed.
+nextInt(): int	Returns a random int value.
+nextInt(n: int): int	Returns a random int value between 0 and n (exclusive).
+nextLong(): long	Returns a random long value.
+nextDouble(): double	Returns a random double value between 0.0 and 1.0 (exclusive).
+nextFloat(): float	Returns a random float value between 0.0F and 1.0F (exclusive).
+nextBoolean(): boolean	Returns a random boolean value.

The Random Class Example

If two Random objects have the same seed, they will generate identical sequences of numbers. For example, the following code creates two Random objects with the same seed 3.

```
Random random1 = new Random(3);  
System.out.print("From random1: ");  
for (int i = 0; i < 10; i++)  
    System.out.print(random1.nextInt(1000) + " ");  
Random random2 = new Random(3);  
System.out.print("\nFrom random2: ");  
for (int i = 0; i < 10; i++)  
    System.out.print(random2.nextInt(1000) + " ");
```

From random1: 734 660 210 581 128 202 549 564 459 961

From random2: 734 660 210 581 128 202 549 564 459 961

The Point2D Class

Java API has a convenient **Point2D** class in the **javafx.geometry** package for representing a point in a two-dimensional plane.

javafx.geometry.Point2D

```
+Point2D(x: double, y: double)
+distance(x: double, y: double): double
+distance(p: Point2D): double
+getX(): double
+getY(): double
+toString(): String
```

Constructs a **Point2D** object with the specified *x*- and *y*-coordinates.
Returns the distance between this point and the specified point (*x*, *y*).
Returns the distance between this point and the specified point *p*.
Returns the *x*-coordinate from this point.
Returns the *y*-coordinate from this point.
Returns a string representation for the point.

TestPoint2D

Thank you
Thank you

